3.The OSI - Layer

What is the OSI Model?

When two computers communicate over a network, an enormous amount of things need to happen — data needs to be broken into pieces, addressed, routed, secured, transmitted as electrical signals, and reassembled at the other end.

The **OSI (Open Systems Interconnection) model** is a **conceptual framework** created by the **ISO (International Organization for Standardization) in 1984** that breaks down this entire communication process into **7 distinct layers**.

Why Was It Created?

In the early days of networking, every company (IBM, DEC, etc.) had their **own proprietary networking systems** that couldn't talk to each other. The OSI model was created to:

- Give everyone a **common language** to describe networking
- Allow **interoperability** between different vendors
- Make it easier to **troubleshoot** — isolate which layer has the problem
- Allow each layer to **evolve independently** without breaking others

> Think of it like a **factory assembly line** — each station does one specific job, passes the product to the next station, and doesn't need to know what the other stations do.

The 7 Layers — Overview

```
Layer 7 ── Application    ← Closest to the user
Layer 6 ── Presentation
Layer 5 ── Session
Layer 4 ── Transport
Layer 3 ── Network
Layer 2 ── Data Link
Layer 1 ── Physical       ← Closest to the wire
```

How Data Flows Through the Layers

This is the most important concept to understand first.

When you send data (say, an HTTP request), it travels **down** all 7 layers on the sender's side, gets transmitted, then travels **up** all 7 layers on the receiver's side.

At each layer going down, a **header (and sometimes trailer) is added** to the data. This is called **Encapsulation**.
At each layer going up, those headers are **stripped off**. This is called **Decapsulation**.

```
SENDER                                    RECEIVER

Application   [  DATA         ]           [  DATA         ]   Application
Presentation  [  DATA         ]           [  DATA         ]   Presentation
Session       [  DATA         ]           [  DATA         ]   Session
Transport     [ TCP | DATA    ]           [ TCP | DATA    ]   Transport
Network       [ IP | TCP|DATA ]           [ IP | TCP|DATA ]   Network
Data Link     [MAC|IP|TCP|DATA|FCS]       [MAC|IP|TCP|DATA|FCS] Data Link
Physical      101010101010101010          101010101010101010  Physical
                      ↓ transmitted as bits ↓
```

Each layer only talks to its **peer layer** on the other side — the Transport layer on your machine communicates logically with the Transport layer on the server, even though physically the data goes through all layers.

Layer 1 — Physical Layer

What It Does

The Physical layer is the **actual hardware** — it deals with the **raw transmission of bits (0s and 1s)** over a physical medium. It defines everything about the physical connection:

- What kind of **cable or wireless signal** is used
- **Voltage levels** that represent 0 and 1
- **Pin layouts** of connectors
- **Transmission speed** (bits per second)
- Whether transmission is **simplex, half-duplex, or full-duplex**

What It Does NOT Do

It has absolutely no understanding of what the bits mean. It just moves them.
The physical road/highway that cars travel on. It doesn't know what's inside the cars — it just provides the path.

Devices at This Layer

- **Hubs** (dumb repeaters — just broadcast bits to all ports)
- **Repeaters** (amplify signal over long distances)
- **Cables** (Ethernet, fiber optic, coaxial)
- **Network Interface Cards (NICs)**

Layer 2 — Data Link Layer

What It Does

The Data Link layer takes raw bits from Layer 1 and organizes them into meaningful units called **frames**. It is responsible for:

- **Framing** — Packaging bits into frames with a defined start and end
- **MAC Addressing** — Every network device has a **MAC (Media Access Control) address** — a hardware-level unique identifier (48 bits, written as `AA:BB:CC:DD:EE:FF`). Layer 2 uses MAC addresses for **local delivery** (within the same network)
- **Error Detection** — Adds a **FCS (Frame Check Sequence)** / CRC at the end of each frame to detect if bits were corrupted during transmission
- **Flow Control** — Prevents a fast sender from overwhelming a slow receiver
- **Media Access Control** — Decides who gets to transmit on a shared medium (avoids collisions)

Two Sub-layers

The Data Link layer is split into two sub-layers:

- **LLC (Logical Link Control)** — Handles flow control and error checking, interfaces with Layer 3
- **MAC (Media Access Control)** — Handles MAC addressing and access to the physical medium

Real World Analogy

The local postal system within a city. MAC addresses are like house addresses within a neighbourhood — they only make sense locally, not globally.

Devices at This Layer

- **Switches** (smart — forward frames only to the correct port based on MAC address table)
- **Bridges**
- **NICs (at the MAC sublayer)**

ARP — A Critical Protocol Here

When your computer wants to send data to `192.168.1.5` on the local network, it knows the IP but needs the MAC address to create a frame. So it **broadcasts** an ARP request:

```
"Hey everyone — who has IP 192.168.1.5? Tell me your MAC address!"
192.168.1.5 replies: "That's me! My MAC is AA:BB:CC:DD:EE:FF"
```

This IP→MAC mapping is cached in the **ARP table**.

Layer 3 — Network Layer

What It Does

The Network layer is responsible for **end-to-end delivery across multiple networks** — this is where **routing** happens. Key responsibilities:

- **Logical Addressing (IP Addressing)** — Assigns and uses IP addresses (not MAC addresses) for global identification
- **Routing** — Determines the **best path** for packets to travel from source to destination across multiple networks/routers
- **Packet Forwarding** — Moves packets hop-by-hop toward the destination
- **Fragmentation** — If a packet is too large for a network segment, Layer 3 breaks it into smaller fragments
- **TTL (Time To Live)** — Each packet has a TTL value that decrements at each router hop, preventing packets from circling forever

Data Unit

Packet

Real World Analogy

The national/international postal system. IP addresses are like full postal addresses including city, state, country — they work globally, not just locally. Routers are like sorting facilities that look at the destination and decide which direction to forward the package.

Devices at This Layer

- **Routers** — read IP addresses, make routing decisions
- **Layer 3 Switches** — switches that can also route
- **Firewalls** (partly operate here)

IP vs MAC — Why Both?

This confuses a lot of people. Here's the key insight:

```
MAC Address → LOCAL delivery (within one network segment)
IP Address  → GLOBAL delivery (across the entire internet)

When a packet travels from your laptop to Google:
- The IP addresses (source & destination) NEVER change
- The MAC addresses change at EVERY hop (router to router)
```

Each router strips the old frame (old MAC), reads the IP, decides next hop, and wraps a new frame (new MAC) to send to the next router.

Layer 4 — Transport Layer

What It Does

The Transport layer is arguably the **most important layer for system design**. It provides **end-to-end communication between processes** (not just devices). Key responsibilities:

- **Port Numbers** — Identifies which application/process the data belongs to. IP gets it to the right device, port gets it to the right application.
- **Segmentation & Reassembly** — Breaks large data into **segments**, numbers them, and reassembles them in order at the destination
- **Connection Management** — Establishes, maintains, and terminates connections (TCP)
- **Reliability** — Guarantees delivery, detects lost packets and retransmits (TCP)
- **Flow Control** — Controls how much data is sent to prevent overwhelming receiver
- **Congestion Control** — Slows down transmission when the network is congested
- **Multiplexing** — Multiple applications can send/receive simultaneously using different ports

Data Unit

Segment (TCP) / **Datagram** (UDP)

The Two Big Protocols — TCP vs UDP

TCP (Transmission Control Protocol)

- **Connection-oriented** — establishes connection first (3-way handshake)
- **Reliable** — guarantees every segment arrives and in order
- **Flow & congestion control** — smart about not overwhelming the network
- **Slower** due to overhead
- **Used for**: HTTP/HTTPS, emails, file transfers — anything where data integrity matters

UDP (User Datagram Protocol)

- **Connectionless** — just fires packets, no handshake
- **Unreliable** — no guarantee of delivery or order
- **No flow control** — as fast as possible
- **Much faster and lower latency**
- **Used for**: Video streaming, gaming, DNS, VoIP — anything where speed matters more than perfect delivery

```
TCP  →  Like a phone call — you establish a connection, confirm
         the other person is there, speak, and hang up properly.

UDP  →  Like shouting across a room — you just send the message
         and hope they hear it. Fast but no guarantee.
```

Port Numbers

```
Well-known ports (0–1023):
  HTTP    → 80
  HTTPS   → 443
  FTP     → 21
  SSH     → 22
  DNS     → 53
  SMTP    → 25
  MySQL   → 3306

Registered ports (1024–49151): used by applications
Dynamic/ephemeral ports (49152–65535): temporary client ports
```

So when you visit google.com, your laptop uses a random ephemeral port (say `52341`) and connects to Google's server on port `443`. The full connection is identified as:

```
192.168.1.5:52341  ──►  142.250.195.46:443
```

This 4-tuple (src IP, src port, dst IP, dst port) uniquely identifies every connection.

Layer 5 — Session Layer

What It Does

The Session layer manages **sessions** — it establishes, maintains, and terminates **dialogues** between two applications. Think of a session as an ongoing conversation. Key responsibilities:

- **Session Establishment** — Sets up a session before communication begins
- **Session Maintenance** — Keeps the session alive during long transfers
- **Session Termination** — Cleanly closes sessions when done
- **Synchronization** — Adds **checkpoints** in long data transfers so if a connection fails, transfer can resume from the last checkpoint rather than starting over
- **Dialog Control** — Manages who can send data and when (half-duplex vs full-duplex)
- **Authentication & Authorization** — Some session-level security (though mostly handled at higher layers today)

Real World Analogy

A phone call management system — it dials the call, keeps it connected, handles "hold" and reconnection if dropped, and hangs up properly.

In modern internet architecture, the Session layer functionality is often **absorbed into the Application layer** (HTTP sessions, cookies, JWT tokens, Web Sockets all handle session management at Layer 7). The Session layer is less distinctly visible in TCP/IP but the concepts are very relevant.

Layer 6 – Presentation Layer

What It Does

The Presentation layer is the **translator** – it ensures that data sent by the application on one system can be understood by the application on another system. Key responsibilities:

- **Translation/Encoding** – Converts data between different formats. For example, converting character encodings (ASCII → Unicode)
- **Encryption & Decryption** – Encrypts data before transmission, decrypts on arrival. **TLS/SSL operates here**
- **Compression & Decompression** – Reduces data size for efficient transmission (gzip, JPEG compression, MP3 encoding)
- **Serialization** – Converting data structures into a format that can be transmitted (JSON, XML)

Real World Analogy

A translator at the United Nations – both parties speak different languages, the translator converts so both understand each other perfectly.

Like the Session layer, Presentation layer functions are largely handled **within application-layer protocols** in modern systems. TLS (which is technically Presentation layer) is critically important – it's what makes HTTPS secure.

Layer 7 – Application Layer

What It Does

The Application layer is the **closest layer to the end user** – it provides network services directly to applications. This is what users and developers interact with most. Key responsibilities:

- **Providing network services to applications** – email, web browsing, file transfer, DNS lookup
- **User authentication**
- **Data formatting for user consumption**
- **Identifying communication partners**

Important: The Application layer is NOT the application itself (Chrome, Outlook). It's the **protocols and services** that applications use to communicate over the network.

Main Protocols – These Are Critical for System Design

Protocol	Port	Description
HTTP	80	Web communication – request/response
HTTPS	443	Secure HTTP (HTTP + TLS)
DNS	53	Domain name resolution
SMTP	25/587	Sending emails
IMAP	993	Receiving emails (syncs)
POP3	110/995	Receiving emails (downloads)
FTP	21	File transfer
SFTP	22	Secure file transfer (over SSH)
SSH	22	Secure remote login
WebSocket	80/443	Full-duplex real-time communication
gRPC	varies	High-performance RPC framework
SNMP	161	Network device management
DHCP	67/68	Dynamic IP assignment

All 7 Layers Together — The Complete Picture

Layer	Name	Protocols / Examples	Data Unit
7	Application	HTTP, HTTPS, DNS, SMTP, FTP, SSH	Message
6	Presentation	TLS/SSL, JPEG, JSON, gzip, Unicode	Data
5	Session	NetBIOS, RPC, SIP, PPTP	Data
4	Transport	TCP, UDP	Segment
3	Network	IP, ICMP, BGP, OSPF, IPSec	Packet
2	Data Link	Ethernet, Wi-Fi, ARP, VLAN, STP	Frame
1	Physical	Cables, Wi-Fi signals, Bluetooth	Bits

OSI vs TCP/IP Model

In practice, the internet runs on the **TCP/IP model**, not OSI. But OSI is used for **teaching and troubleshooting** because it's more detailed.

```
OSI Model          TCP/IP Model
─────────          ────────────
Application  ┐
Presentation ├──► Application   (HTTP, DNS, SMTP, FTP)
Session      ┘
Transport    ───► Transport     (TCP, UDP)
Network      ───► Internet      (IP, ICMP, BGP)
Data Link    ┐
Physical     ┴──► Network Access (Ethernet, Wi-Fi)
```

The TCP/IP model collapses the top 3 OSI layers into one Application layer, and the bottom 2 into one Network Access layer.

How This Applies to System Design

Layer	System Design Relevance
Layer 7	API design, HTTP methods, WebSockets, gRPC
Layer 6	TLS/HTTPS, data serialization (JSON vs Protobuf), compression
Layer 5	Session management, connection pooling
Layer 4	TCP vs UDP choice, load balancing (L4 vs L7), ports
Layer 3	IP routing, VPCs in cloud, firewalls, VPNs
Layer 2	Internal data center networking, VLANs
Layer 1	Physical infrastructure (rarely discussed in interviews)

🌐 Example: You open amazon.com in your browser

Let's trace what happens **layer by layer** as your request goes from your laptop → Amazon server → back to you.

Data Flow Through OSI Layers (Real Example)

7. Application Layer (User Interaction)

- You type: `amazon.com` in Chrome
- Browser creates an **HTTP request**:

```
GET / HTTP/1.1
Host: amazon.com
```

6. Presentation Layer (Formatting & Security)

- Browser checks: "Is this HTTPS?"
- It **encrypts the request using SSL/TLS**
  👉 Now data becomes:

> Encrypted unreadable format (for security)

5. Session Layer (Connection Management)

- A **session is established** with Amazon's server
- TLS handshake happens:
  - Exchange keys
  - Agree on encryption
    👉 Think:

    > "Let's agree on how we'll talk securely"

4. Transport Layer (Reliable Delivery – TCP)

- Data is broken into **segments**
- Uses **TCP**:
  - Adds **port numbers** (e.g., 443 for HTTPS)
  - Ensures:
    - Order
    - No data loss
    - Retries if needed
      👉 Think:

      > "Send this safely and in order"

3. Network Layer (Routing – IP)

- Adds:
  - Your IP (source)
  - Amazon server IP (destination)
- Decides:

  > "Which path should this packet take across the internet?"
  > 👉 Routers operate here

2. Data Link Layer (Local Delivery – MAC)

- Converts packet → **frame**
- Adds:
  - Your device MAC address
  - Router MAC address
    👉 Think:

    > "Send this to the next hop (your WiFi router)"

1. Physical Layer (Actual Transmission)

- Data becomes:
  - Electrical signals (Ethernet)
  - Radio waves (WiFi)

👉 Actually travels through:

- Wi Fi signals
- Cables
- Fiber optics

🔁 Now at Amazon Server (Reverse Happens)

Amazon server receives it:
1 → Converts signals to data
2 → Checks MAC
3 → Reads IP
4 → Reassembles TCP segments
5 → Maintains session

6 → Decrypts data
7 → Web server reads HTTP request

Server Response

Amazon server sends back:

- HTML
- CSS
- JS

👉 Same layers again (reverse flow) back to you.

⚡ Final Result

Your browser:

- Receives data
- Renders UI
  👉 You see Amazon homepage

Real Engineering Insight (Important)

When you debug systems:

- Timeout? → Transport layer issue
- SSL error? → Presentation layer
- 404 error? → Application layer
- Packet loss? → Network/Physical